Redes de Comunicação — Folha de revisão

Gonçalo Sousa Capítulos 1–2. As receitas do resumo, sem exemplos: para rever antes do teste.

Comutação

Circuitos: recursos reservados extremo-a-extremo antes de comunicar (FDM: banda por utilizador; TDM: *slot* fixo por ciclo). Desempenho garantido, recursos parados quando inativos. Capacidade do circuito = R/nº de slots; tempo total = estabelecimento + $F/R_{circuito}$.

Pacotes: sem reserva, multiplexagem estatística, capacidade total da ligação por pacote, filas e congestionamento. *Store-and-forward*: o *router* só reenvia depois de receber o pacote todo → N saltos iguais: $N \cdot L/R$ (+ propagações).

Quantos utilizadores: circuitos = R/r; pacotes: $X \sim \text{Bin}(N, p)$, $P(X > k) = 1 - \sum_{i=0}^{k} \binom{N}{i} p^i (1-p)^{N-i}$ tem de ser desprezável.

Atrasos e débito

Componente	Fórmula	Depende de
d_{proc}	—	verificar cabeçalho, escolher saída (μs)
d_{fila}	—	congestionamento nesse instante
d_{trans}	L/R	tamanho do pacote e capacidade, **não** da distância
d_{prop}	l/v	distância e meio ($v \approx 2 \times 10^8$ m/s), **não** do pacote

n **pacotes por ligações em série**: o último chega em $n \cdot d_{trans}$ **da ligação mais lenta** + um d_{trans} de cada outra + todas as d_{prop}. Traçar à mão: início na ligação seguinte = máx(chegada ao *router*, fim do pacote anterior nessa ligação). No teste (sem calculadora) desenha-se: faixas de largura d_{trans}, inclinação d_{prop}.

Intensidade de tráfego La/R: ≈ 0 fila vazia; → 1 atraso explode (não linear); > 1 atraso → ∞ (perdas).

Débito = mín das capacidades do percurso (ligação de estrangulamento); com n fluxos a partilhar R: $\min(R_s, R_c, R/n)$. **Produto largura de banda-atraso** $R \cdot d_{prop}$ = bits em trânsito. **Comprimento de um bit** = v/R.

traceroute: 3 sondas por salto; salto grande de RTT = ligação longa (propagação); $* * *$ = sem resposta, não é rede cortada.

Camadas

Camada	Unidade	Cabeçalho	Dispositivo
Aplicação	mensagem M	—	terminais
Transporte	segmento C_tM	portas	terminais
Rede	datagrama C_nC_tM	IPs	*router* (só lê C_n)
Ligação	trama $C_lC_nC_tM$	MACs	*switch* (só lê C_l)
Física	bits	—	—

MAC muda a cada salto; IP mantém-se. OSI = 7 camadas (+ apresentação e sessão), que na Internet ficam dentro da aplicação.

Aplicações e transporte

Cliente-servidor: servidor sempre ligado, IP fixo; clientes não falam entre si. **P2P**: sem servidor, pares intermitentes, escalável, difícil de gerir. **Híbridas**: BitTorrent (*tracker*), mensagens instantâneas (presença central). Processo = (IP, porta).

	TCP	UDP
conexão	sim (*handshake*)	não
fiável, por ordem	sim	não
controlo de fluxo e congestionamento	sim	não
delineação de mensagens	não (fluxo de bytes)	sim (1 datagrama = 1 mensagem)
garantias de atraso/débito	não	não

TLS: cifragem + integridade + autenticação, na camada de aplicação, sobre TCP.

HTTP

Tempos (RTT = ida e volta de um pacote pequeno; T = transmissão de um objeto; página = 1 HTML + n objetos):

Modo	Tempo
não persistente, em série	$(n+1)(2\,\mathrm{RTT} + T)$
persistente sem *pipelining*	$2\,\mathrm{RTT} + n\,\mathrm{RTT} + (n+1)T$
persistente com *pipelining*	$2\,\mathrm{RTT} + 1\,\mathrm{RTT} + (n+1)T$

HTTP/1.0 fecha após cada resposta (salvo `Connection: keep-alive`); **HTTP/1.1** persistente por omissão (`Connection: close` fecha), `Host:` obrigatório. Fim de cada resposta: `Content-Length`. *Stateless*.

Pedido: `método URL versão` / cabeçalhos / linha vazia / corpo. Métodos: GET, POST, HEAD (1.0); PUT, DELETE (1.1). **Códigos**: 200 OK, 301 Moved Permanently (`Location:`), 400 Bad Request, 404 Not Found, 505 Version Not Supported.

HTTP/2: binário, tramas intercaladas (sem bloqueio HOL), prioridades, *server push*. **HTTP/3**: QUIC sobre UDP, 1 *handshake* para transporte + TLS, 0-RTT com *ticket* de sessão.

Proxy: cliente e servidor; menos tempo de resposta e tráfego de acesso.

Wireshark: conexão = par de portas; abre com SYN, fecha com FIN; simultâneas se se sobrepõem; persistente se > 1 pedido antes do FIN.

FTP, email

FTP: controlo na porta 21 (*out-of-band*, com estado), dados a partir da 20, uma conexão por ficheiro. `USER`, `PASS`, `PORT`, `LIST`, `RETR`, `STOR`. `PORT a,b,c,d,p1,p2` → IP a.b.c.d, porta $256p_1 + p_2$. `PASV`: o cliente abre a conexão de dados.

SMTP (porta 25, TCP, *push*, persistente, ASCII 7 bits): `HELO/EHLO` → `MAIL FROM` → `RCPT TO` (um por destinatário) → `DATA` → mensagem (cabeçalhos, linha vazia, corpo) → `.` → `QUIT`. Envelope (`RCPT TO`) ≠ cabeçalho (`To:`). `250-` = há mais linhas. Binário/acentos: MIME (`Content-Type`, `Content-Transfer-Encoding: base64`). STARTTLS → repetir `EHLO`. O remetente não é verificado (SPF, DKIM, DMARC à parte).

Acesso: POP3 (`user`, `pass`, `list`, `retr`, `dele`, `quit`; "transfere e apaga"; sem estado entre sessões), IMAP (mensagens e pastas no servidor, com estado), webmail (HTTP).

DNS

UDP, binário, pergunta e resposta com o mesmo formato. Cabeçalho: `id` (associa resposta à pergunta), *flags* `qr` (resposta), `rd` (recursão pedida), `ra` (disponível), `aa` (com autoridade), 4 contadores. Secções: *question*, *answer*, *authority* (NS), *additional* (A dos NS).

RR	nome	valor
A / AAAA	máquina	IPv4 / IPv6
NS	domínio	servidor com autoridade
CNAME	pseudónimo	nome canónico
MX	domínio	servidor de email (preferência: menor primeiro)

Hierarquia: raiz (13, *root hints*) → TLD → com autoridade. Servidor local: recebe pedidos **recursivos**, faz pedidos **iterativos**, guarda em *cache*. Caso normal com *cache* vazia: 8 mensagens; só a parte iterativa: 6 setas, todas de e para o cliente. **TTL**: fixo no servidor com autoridade; desce quando vem da *cache*. Registar um domínio: NS + A do servidor de nomes no TLD; A, MX, CNAME… no servidor com autoridade.

`dig nome TIPO`, `@servidor`, `+short`, `+norecurse` (iterativo), `+trace`, `+qr`; `host -t mx`.

P2P

$$d_{CS} = \max\left\{\frac{NF}{u_s},\ \frac{F}{d_{\min}}\right\} \qquad d_{P2P} = \max\left\{\frac{F}{u_s},\ \frac{F}{d_{\min}},\ \frac{NF}{u_s + \sum u_i}\right\}$$

d_{CS} cresce linearmente com N; d_{P2P} fica limitado. **BitTorrent**: pedaços de 256 kB; *rarest first*; *tit-for-tat* (top 4 a cada 10 s + 1 par ao acaso a cada 30 s).

Sockets

TCP: servidor `ServerSocket(porta)` → `accept()` dá uma socket nova por cliente; cliente `Socket(host, porta)` liga logo. Mensagem = linha (`\n`). **UDP**: `DatagramSocket`; cada `DatagramPacket` leva IP e porta; o servidor responde para `getAddress()`/`getPort()`; usar `getLength()` ao converter os dados.

Persistente: `while ((l = in.readLine()) != null)`; `null` = o outro lado fechou (CTRL-D no teclado). Iterativo + persistente: o 2.º cliente liga-se (fica na fila do `accept()`) mas só é lido quando o 1.º fecha. *Threads*: `new Thread(new Helper(sock)).start()` e voltar ao `accept()`.

Cliente SMTP: linhas com `\r\n`; ler a resposta até uma linha sem – na posição 3 e verificar o código (2xx ok, 3xx continua, 4xx temporário, 5xx definitivo); `From:`/`To:`/`Subject:` vão dentro do `DATA`; corpo com `.` inicial → `..`.

Servidores: iterativo (um de cada vez); concorrente com processos (`fork()`) ou *threads* (leve, menos robusto); multiplex (`select()`, um só processo, não usa vários *cores*); *pre-forking/pre-threaded* (carga alta de pedidos curtos).

	HTTP	FTP	SMTP	POP3/IMAP	DNS
transporte	TCP	TCP	TCP	TCP	UDP
porta	80	21 (+20)	25	110/143	53
estado	não	sim	—	POP3 não; IMAP sim	—
controlo	*in-band*	*out-of-band*	*in-band*	*in-band*	—
sentido	*pull*	ambos	*push*	*pull*	pergunta/resposta